

Answer to a Leetcode question. The answer Code generated by this coding agent

The question from LeeTCode:

Write a program that, Given a string s, find the length of the longest substring without duplicate characters.

Example 1: Input: s = "abcabcbb" Output: 3 Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2: Input: s = "bbbbbb" Output: 1 Explanation: The answer is "b", with the length of 1

(additional part to the prompt) the program should have a window where the user input a string and the output of the program is: a print of the length of the longest substring without duplicate, and a print of the longest substring.

The Coding agent answer:

Certainly! Below is a Python program that accomplishes the task described. It uses a sliding window approach to find the longest substring without duplicate characters.

```
def length_of_longest_substring(s):
    """
    Finds the length of the longest substring without duplicate
    characters.

    :param s: Input string
    :return: Length of the longest substring without duplicates and the
    substring itself
    """
    # Dictionary to store the last seen index of each character
    char_index_map = {}
    start = 0
    max_length = 0
    max_substring = ""

    for end in range(len(s)):
        if s[end] in char_index_map:
            # Update the start to the right of the last seen index of the
            current character
            start = max(start, char_index_map[s[end]] + 1)

        # Update the maximum length and substring if the current substring
        is longer
        if end - start + 1 > max_length:
```

```

        max_length = end - start + 1
        max_substring = s[start:end+1]

    # Store the last seen index of the current character
    char_index_map[s[end]] = end

    return max_length, max_substring

def main():
    # Get user input
    s = input("Enter a string: ")

    # Find the length of the longest substring without duplicate
    # characters
    length, substring = length_of_longest_substring(s)

    # Print the results
    print(f"Length of the longest substring without duplicate
characters: {length}")
    print(f"The longest substring without duplicate characters:
'{substring}'")

if __name__ == "__main__":
    main()

```

The coding agent explained the code too:

Explanation:

1. **Initialization**:

- `char\_index\_map` is a dictionary to store the last seen index of each character.
- `left` is the starting index of the current window.
- `max\_length` keeps track of the maximum length of the substring found so far.
- `start` records the starting index of the longest substring.

2. **Sliding Window**:

- Iterate through the string using the `right` pointer.
- If the current character is already in the dictionary and its last seen index is within the current window (`char\_index\_map[s[right]] >= left`), update the `left` pointer to be one position right of the last seen index of the current character.
- Update the last seen index of the current character in the dictionary.
- Calculate the length of the current window (`right - left + 1`).

- If the current window is longer than the previously recorded maximum length, update `max\_length` and record the starting index of this window.

3. ****Extracting the Longest Substring****:

- After iterating through the string, extract the longest substring using the recorded `start` index and `max\_length`.

4. ****User Interaction****:

- The program prompts the user to enter a string.
- It then calculates and prints the length of the longest substring without duplicate characters and the substring itself.

This implementation ensures that the program runs efficiently with a time complexity of $O(n)$, where n is the length of the input string.

Running the generated code:

Enter a string: jkjkfgfgf

Length of the longest substring without duplicate characters: 4

'The longest substring without duplicate characters: 'jkfg'